

# Segnali, pipe, FIFO

Lezione 14

Sistemi Operativi - II Modulo  
(Canale A - L)

**Prof. Paolo Zuliani**

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



**SAPIENZA**  
UNIVERSITÀ DI ROMA



# Agenda

- Segnali
- Gestione dei segnali
- Handler sicuri - gestione segnali - sezione critica
- Esecuzione di handler dei segnali - `signal()` e `sigaction()`
- Altre system call utili per i segnali
- Pipe e FIFO



# Segnali

- Sono interruzioni software
- Inviati dal kernel ad un processo
- Inviati da un processo ad un altro processo



# Segnali

- Condizioni anomale che si verificano generano INTERRUPTS ovvero il kernel invia un segnale ai processi in esecuzione.
- Condizioni anomale possono essere:
  1. Richiesta di sospensione da parte dell'utente (Control-Z = SIGSTOP)
  2. Richiesta di terminazione da parte dell'utente (Control-C = SIGINT)
  3. Una eccezione hardware (divisione per 0; riferimento di memoria non valido; etc...)

# Segnali

- Un segnale può essere generato anche da condizioni NON anomale:
  - La terminazione di un processo figlio
  - Un cronometro impostato con `alarm` scade
  - Un segnale inviato da un processo ad un altro mediante la `syscall kill`
  - Un segnale mandato da terminale mediante il comando `kill`
  - Un dato arriva su di una connessione di rete (SIGURG)
  - Un processo scrive su di una pipe che non ha un "lettore" (SIGPIPE)
  - ...

# Segnali

- La lista completa dei segnali (costanti intere) è in `<signal.h>`  
(in `/usr/include/signal.h` per debian)
- Ad ogni evento è associato un segnale

# I principali segnali ed eventi sono: (man 7 signal)

|         |                                                          |
|---------|----------------------------------------------------------|
| SIGINT  | 2 terminazione <CTRL+C> da tastiera                      |
| SIGQUIT | 3 Core dump Uscita                                       |
| SIGILL  | 4 Core dump Istruzione illegale                          |
| SIGABR  | 6 Core dump Abort                                        |
| SIGFPE  | 8 Core dump Eccezione di tipo aritmetico                 |
| SIGKILL | 9 terminazione Kill (non gestibile)                      |
| SIGUSR1 | 10 terminazione Definito dall'utente                     |
| SIGSEGV | 11 Core dump Segmentation Fault                          |
| SIGUSR2 | 12 terminazione Definito dall'utente                     |
| SIGPIPE | 13 terminazione Scrittura senza lettori su pipe o socket |
| SIGALRM | 14 Terminazione Allarme temporizzato                     |
| SIGTERM | 15 terminazione Terminazione software                    |
| SIGCHLD | 17 ignorato Status del figlio cambiato                   |
| SIGSTOP | 19 Stop Sospende del processo (non gestibile)            |
| SIGTSTP | 20 stop Stop da tastiera                                 |
| SIGTTIN | 21 stop Lettura su tty in background                     |
| SIGTTOU | 22 stop Scrittura su tty in background                   |



# Gestione dei Segnali

- I segnali sono un esempio di eventi *asincroni*:
  - occorrono ad un tempo non predeterminato
  - il processo deve dire al kernel cosa fare se e quando l'evento occorre
- "Cosa fare" è detto azione associata al processo, e sono tre:
  1. IGNORARE IL SEGNALE (*Ignore*): si può fare con tutti tranne che con SIGKILL e SIGSTOP. [[Perché????]]
  2. CATTURARE IL SEGNALE (*Catch*): il processo chiede al kernel di eseguire una funzione definita dal programmatore (Signal handler o gestore del segnale). I segnali SIGKILL e SIGSTOP non possono essere catturati
  3. ESEGUIRE L'AZIONE PREDEFINITA: ad ogni segnale è associato un sdefault handler

# Gestione dei Segnali

- Handler di default sono (vedi fig 10.1 su APUE per la lista completa):
  - Termina il processo e genera il *core dump* (file core)
  - Termina il processo senza generare il core dump
  - Ignora e rimuovi il segnale
  - Sospende il processo
  - Riesuma il processo
- Su APUE c'è una descrizione dettagliata per ogni segnale

# Handler, gestione segnali, sezione critica

Quando un processo riceve un segnale che deve gestire con un handler:

1. Interrompe il proprio flusso di esecuzione
2. Esegue l'handler associato al segnale
3. Riprende l'esecuzione dal punto in cui era stato interrotto

Tali passi possono essere realizzati con:

```
/* funzione di libreria*/  
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);  
  
/* system call */  
int rt_sigprocmask(int how, const kernel_sigset_t *set,  
kernel_sigset_t *oldset, size_t sigsetsize);
```



# sigprocmask ( )

- Consente di leggere/impostare la maschera segnali (ci dice i segnali bloccati)
- Un segnale bloccato non viene perso ma considerato 'pendente' (*pending*) e consegnato al processo quando viene sbloccato
- Ad eccezione di SIGCHLD, istanze multiple di segnali pendenti non vengono considerate

```
int sigprocmask(int how, const
sigset_t *set, sigset_t *oldset)
```

- `how` dice come gestire il segnale e può assumere i seguenti valori
  - `SIG_BLOCK`: blocca i segnali definiti in `set`
  - `SIG_UNBLOCK` sblocca i segnali definiti in `set`
  - `SIG_SETMASK` imposta la maschera a `set`
- `set` è la maschera da usare
- `old_set` è la maschera prima dell'invocazione della funzione. Utile per ripristinare la maschera dopo la chiamata
- vediamo `sigprocmask.c`

# `sigprocmask ( )`

- E' applicabile solo a processi e NON a threads (che vedremo più avanti) perché ognuno ha una sua maschera
- SIGKILL e SIGSTOP non possono essere bloccati
- Ogni processo creato tramite `fork ( )` eredita maschera segnali
- La maschera dei segnali è mantenuta dopo `exec ( )`

# Esercizi su `sigprocmask.c`

1. aggiungere tutti i controlli mancanti sui possibili errori delle syscall. In caso di errore stampate un messaggio su `stderr` e poi chiamate `exit(EXIT_FAILURE)`. Nel messaggio di errore potete consultare `errno` (vedi lezione 10).
2. scrivere una funzione `print_set` che prenda un `sigset_t` e stampi i segnali in esso contenuti (usare `sigpending()` e `sigismember()` dentro un ciclo `for`).
3. Modificare il codice in modo tale che ogni 5 secondi controlli se l'utente ha premuto CTRL+C o CTRL+Z o abbia inviato un SIGUSR1, e in tal caso si limiti a scrivere sullo `stdout` tale evento. [Suggerimento: usate un ciclo infinito e `sleep()`.]

# Esecuzione di handler dei segnali - `signal()` e `sigaction()`

*(L'USO DELLA SIGNAL E' SCONSIGLIATO PERCHE' L'IMPLEMENTAZIONE NON E' STANDARD E PUO' VARIARE DA SISTEMA A SISTEMA)*

`sighandler_t signal(int signum, sighandler_t handler)`

- Imposta il gestore del segnale `signum` alla funzione `handler`
- Ritorna: `SIG_ERR` o il valore del precedente gestore
- Sono definite 2 macro
  - `SIG_IGN` ignora il segnale
  - `SIG_DFL` assegna il gestore di default. Utile per ripristinare il gestore del segnale a default
- vedere `signal.c`



```
int sigaction(int signum, const struct
sigaction *act, struct sigaction *oldact)
```

- signum: **segnale**
- act: **gestore del segnale** (vedi struct sigaction) **se act=NULL è usato oldact**
- oldact: **gestore precedente**

```
struct sigaction {
    void (*sa_handler)(int);    /*Puntatore alla funzione signal
                                handler. Può essere SIG_IGN, SIG_DFL
                                o puntatore a funzione.* /
    void (*sa_sigaction)(int, siginfo_t *, void *); /*Alternativo
                                                    a sa_handler*/
    sigset_t sa_mask;           /*Specifica la maschera dei segnali che
                                dovrebbero essere bloccati durante
                                l'esecuzione dell'handler*/
    int sa_flags;               /*Flags per modificare il comportamento del
                                segnale*/
    void (*sa_restorer)(void); /*obsoleto*/};
```



# Altre system calls utili

- `int kill(pid_t pid, int sig);`
- `unsigned int alarm(unsigned int seconds);`
  - invia un SIGALARM dopo seconds secondi
- `int pause(void);`
  - il processo chiamante rimane bloccato finché un segnale non viene ricevuto
  - vedi `pause.c`
- `int sigpending(sigset_t *set);`
  - restituisce in set i segnali pendenti
- `int sigsuspend(const sigset_t *mask);`
  - sospende il processo che invoca la call e rimpiazza la maschera con mask
  - il processo rimane sospeso finché non arriva un segnale per cui è definito un gestore o un segnale di terminazione
  - vedi `sigsuspend.c`

# Inter Process Communication (IPC)

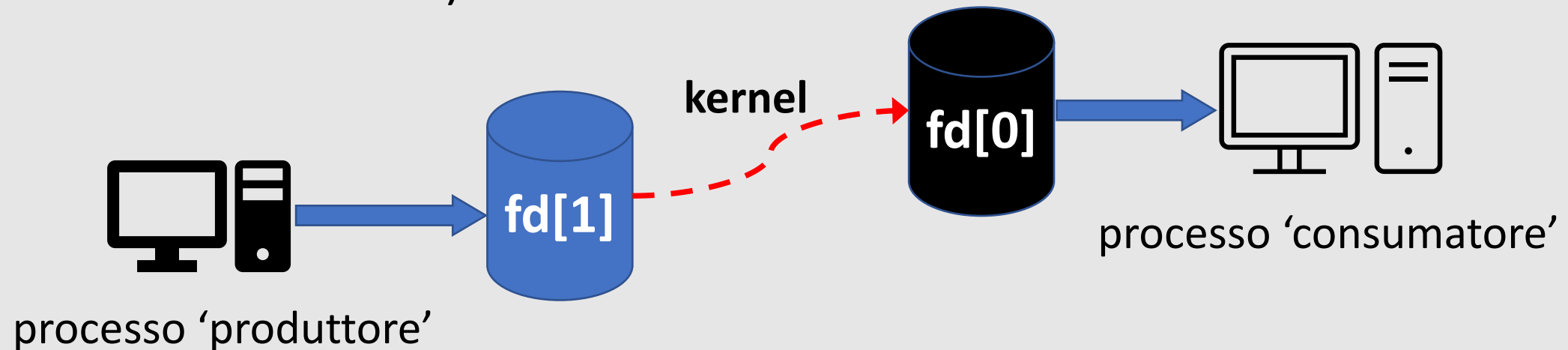


# Pipe e FIFO

- Unix ha due tipi di IPC: *FIFO* (o *named pipe*) e *pipe* (o *unnamed pipe*)
- La scrittura dei dati su una pipe/fifo e quindi anche la lettura avviene in maniera sequenziale (First In First Out)
- La **FIFO** è uno **speciale tipo di file** che può essere creato dalle system call `mkfifo` o `mknod`. Può essere utilizzata da più processi, **full-duplex**
- La **pipe** è una **struttura dati in memoria**, è **half-duplex**. La creazione della pipe può essere effettuata con la system call `pipe` che crea due file descriptor (uno in lettura ed uno in scrittura)
- *Esempio*: un processo crea una pipe, poi crea un figlio e usa la pipe per comunicare con il figlio che eredita i descrittori dei file
- La pipe può anche essere usata tra 2 processi che hanno un antenato comune

```
int pipe(int pipefd[2])
```

- Se tutto OK, `pipe` crea due file descriptors e ritorna 0  
    `pipefd[0]`: file descriptor di lettura  
    `pipefd[1]`: file descriptor di scrittura
- In caso di problemi, `pipe` ritorna -1 e imposta `errno` (`pipefd` non viene modificato)



```
int pipe(int pipefd[2])
```

I dati scritti sulla pipe sono «bufferizzati» dal kernel finché non sono letti. Le pipe hanno una *dimensione massima*:

- Se un processo legge da una pipe vuota allora rimane **bloccato**
- Se un processo scrive su una pipe piena allora rimane **bloccato**
- Una pipe viene chiusa quando tutti e due i processi hanno invocato la close
- Operazioni di lettura (read) su una pipe il cui fd di scrittura è stato chiuso con close ritorna 0
- Operazioni di scrittura (write) su una pipe il cui fd di lettura è stato chiuso con close ritornano -1 e ricevono il segnale SIGPIPE
- **vedi:** `pipe.c`, `pipequater.c` e `pipe_quintes.c`



```
int mkfifo(const char *pathname,  
mode_t mode)
```

- Crea una FIFO con nome `pathname` e modalità di accesso `mode`.
- Una volta creata una FIFO, può essere acceduta da qualsiasi processo (che abbia i diritti per l'accesso al file) in lettura/scrittura, tuttavia, le operazioni devono essere *simultanee*. Un processo che apre la FIFO in lettura rimane bloccato finché non c'è nessun processo che la apre in scrittura (e viceversa).
- `mkfifo` ritorna 0 in caso di successo e -1 in caso di errore impostando `errno` di conseguenza.
  - Una volta creata, la FIFO può essere gestita (open, read write e close) come un qualsiasi altro file
  - È possibile aprire una FIFO in maniera *non bloccante* passando l'attributo `O_NONBLOCK` alla system call `open`

# Domande?

